

Simulador Máquina de Turing

INFO 139 – Teoría de Autómatas

Profesora: María Eliana de la Maza W.

Estudiantes: Samantha Espinoza, Luis Olivares, Fernando Ríos

Fecha de entrega: 15 de junio de 2026

1. Introducción

Las Máquinas de Turing (MT) son el modelo computacional más general que se conoce en la Teoría de Autómatas, propuestas por Alan Turing en 1936. Constituyen la base teórica de la computación moderna y son capaces de reconocer lenguajes recursivamente enumerables, incluyendo lenguajes regulares y lenguajes de libre contexto, además de calcular funciones enteras.

Formalmente, una MT se define como $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, donde Q es el conjunto finito de estados, Σ es el alfabeto de entrada, Γ es el alfabeto de la cinta, δ es la función de transición, q_0 es el estado inicial, B es el símbolo blanco y F es el conjunto de estados finales. La función de transición $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{\text{Izquierda, Derecha}\}$ determina, dado un estado y el símbolo leído en la cinta, el nuevo estado, el símbolo a escribir y la dirección del movimiento del cabezal.

El objetivo de este trabajo fue diseñar e implementar un programa computacional capaz de simular el funcionamiento de cualquier Máquina de Turing determinista, a partir de una descripción formal de sus transiciones, estado inicial y estados de aceptación. El simulador permite ingresar palabras de entrada, indicando si pertenece o no al lenguaje reconocido por la máquina, mostrando además cada descripción instantánea del proceso de verificación.

2. Desarrollo

2.1 Estructuras de datos utilizadas

Siguiendo la definición formal $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ vista en clases, los componentes de la MT se representarán en Python de la siguiente manera:

- **Estado Inicial (q_0):** str (cadena de texto). Almacena el identificador del estado inicial (Ej: "q0"). Permite comparaciones directas durante la ejecución.
- **Estados Finales (F):** set (conjunto de strings). Se utiliza un conjunto para soportar múltiples estados finales. La pertenencia se verifica con la expresión: `estado_actual in estados_finales`.
- **Función de Transición (δ):** dict (diccionario asociativo). Las llaves son tuplas (`estado_actual, simbolo_leido`) y los valores son tuplas (`nuevo_estado, simbolo_escrito, direccion`).
- **Cinta / Palabra (Γ^*):** list (lista mutable de caracteres) junto con un puntero entero (int) para el cabezal. Si el cabezal supera el tamaño actual de la cinta, se añade dinámicamente el símbolo blanco "B" mediante la expresión `.append()`, simulando la naturaleza semi-infinita de la cinta.

2.2 Correspondencia entre los componentes teóricos y las estructuras Python

Componente Teórico MT	Tipo de dato Python	Ejemplo
Estado Inicial (q_0)	str	"q0"
Estados Finales (F)	set	{"q4"}
Transiciones (δ)	dict	{ ("q0","0"): ("q1","X","D") }
Cinta / Palabra (Γ^*)	list	['0','0','1','1','B']
Símbolo Blanco (B)	str (constante)	"B"

2.3 Implementación del Simulador

El núcleo del simulador es la clase **MaquinaDeTuring** definida en el archivo **maquina_turing.py**. Esta clase recibe tres parámetros en su constructor:

- **estado_inicial:** estado q_0 desde el cual comienza la simulación.
- **estados_finales:** conjunto F de estados de aceptación.
- **transiciones:** diccionario que implementa la función δ .

El método **simular(entrada)** implementa el ciclo principal de ejecución de la MT. Su lógica sigue los siguientes pasos:

- Se inicializa la cinta como lista de caracteres de la cadena de entrada (o con un blanco si la cadena es vacía), el cabezal en la posición 0 y el estado en q_0 .
- En cada iteración se registra la Descripción Instantánea (D.I.) actual: estado presente y contenido de la cinta.
- Si el estado actual pertenece a los estados finales, la simulación termina y la cadena es aceptada.
- Se lee el símbolo bajo el cabezal y se busca la transición correspondiente (estado, símbolo) en el diccionario.
- Si no existe transición, la cadena es rechazada.
- Se aplica la transición: se escribe el nuevo símbolo, se actualiza el estado y se mueve el cabezal a la derecha (D) o izquierda (I). Si el cabezal avanza más allá del tamaño de la cinta, se agrega un blanco B.

A continuación, se muestra el código del método **simular()**:

```
def simular(self, entrada):
    cinta = list(entrada) if entrada else [BLANCO]
    cabezal = 0
    estado = self.estado_inicial
    pasos = []

    while True:
        pasos.append("Cinta: " + "".join(cinta) + " estado: " +
estado)
        if estado in self.estados_finales:
            return True, pasos
        simbolo_actual = cinta[cabezal]
        if (estado, simbolo_actual) not in self.tranciciones:
            return False, pasos
        resultado = self.tranciciones[(estado, simbolo_actual)]
        nuevo_estado = resultado[0]
        simbolo_escrito = resultado[1]
        direccion = resultado[2]
        cinta[cabezal] = simbolo_escrito
        estado = nuevo_estado
        if direccion == "D":
            cabezal += 1
            if cabezal >= len(cinta):
                cinta.append(BLANCO)
        else:
            cabezal -= 1
```

3. Especificación del programa

3.1 Archivos

Nombre del archivo	Descripción
maquina_turing.py	Contiene la clase MaquinaDeTuring con la lógica principal, aquí se almacenan los estados, transiciones y se ejecuta la simulación de la cinta.
interfaz.py	Interfaz gráfica de la aplicación, desarrollada con Tkinter . Permite al usuario ingresar el estado inicial, estados finales, transiciones y la palabra a evaluar.

3.2 Lenguaje de programación

Se seleccionó el lenguaje **Python** para desarrollar el programa, concretamente **Python 3**, puesto que es un lenguaje sencillo de trabajar, es flexible y da pie para que el código sea auto explicativo. Además de que cuenta con más facilidades a la hora de implementar interfaces.

3.3 Requerimientos de hardware y software

- **Sistema operativo:** Windows, macOS o Linux
- **Procesador:** 1 GHz o superior
- **Memoria RAM:** Mínimo 2 GB
- **Software requerido:** Python 3.8 o superior, Tkinter (incluido por defecto en Python), editor de texto como por ejemplo VS Code (opcional)

3.4 Requisitos previos para ejecutar el programa (Instalar Python 3)

Windows

- Ir al sitio oficial: <https://www.python.org/downloads/>
- Hacer clic en el botón de descarga Download Python 3.x.x.
- Ejecutar como administrador el instalador descargado.
- Marcar la opción Add Python 3.x to PATH y completar la instalación.
- Verificar en terminal cmd con: `python --versión`

Linux (Ubuntu, Debian, Linux Mint)

- Abrir terminal y ejecutar: `sudo apt update && sudo apt install python3`
- Verificar con: `python3 --version`
- Tkinter suele estar incluido; si no: `sudo apt install python3-tk`

3.5 Ejecución del programa

“**maquina_turing.py**” e “**interfaz.py**” deben estar en la misma carpeta.

Windows

- Abrir la consola cmd, navegar a la carpeta del proyecto y ejecutar:
“**python interfaz.py**” o “**py interfaz.py**”

Para Linux/macOS

- Abrir terminal, navegar a la carpeta del proyecto y ejecutar:
“**python3 interfaz.py**”

4. Funcionamiento del programa

Al ejecutar **interfaz.py** se abre una ventana gráfica dividida en dos etapas funcionales:

Configuración de la MT

- Se ingresa el estado inicial de la MT en el campo **Estado inicial**.
- Se ingresan los estados finales de la MT (separados por coma) en el campo **Estado final**.
- Se ingresan las transiciones (una por línea) con el siguiente formato:
estado_origen, simbolo_leido, estado_destino, simbolo_escrito, direccion
(derecha o izquierda).

Evaluación de palabras

- El usuario ingresa la palabra a evaluar en el campo Palabra.
- Al presionar el botón Mostrar, se ejecuta la simulación.
- El resultado (**ACEPTADA** o **RECHAZADA**) se muestra en pantalla con el color correspondiente (verde o rojo).
- El área inferior muestra las Descripciones Instantáneas de cada paso de la simulación, permitiendo seguir el comportamiento de la máquina.

5. Ejemplo de ejecución

Una MT que reconoce el lenguaje $L = \{ 0^n 1^n / n \geq 1 \}$

Transiciones a ingresar (una por línea):

Estado origen	Símbolo leído	Estado destino	Símbolo escrito	Dirección
q0	0	q1	X	D
q1	0	q1	0	D
q1	Y	q1	Y	D
q1	1	q2	Y	I
q2	0	q2	0	I
q2	Y	q2	Y	I
q2	X	q0	X	D
q0	Y	q3	Y	D
q3	Y	q3	Y	D
q3	B	q4	B	I

Para copiar y probar en el programa:

q0,0,q1,X,D
q0,Y,q3,Y,D
q1,0,q1,0,D
q1,1,q2,Y,I
q1,Y,q1,Y,D
q2,0,q2,0,I
q2,X,q0,X,D
q2,Y,q2,Y,I
q3,Y,q3,Y,D
q3,B,q4,B,I

Estado inicial: q_0

Estado final: q_4

Palabras aceptadas: 01, 0011, 000111, etc.

Palabras rechazadas: 10, 001, 0001, 0111, etc.

5. Conclusiones

El desarrollo de esta tarea, que involucró implementar un simulador de Máquina de Turing, nos permitió comprender de una manera bastante práctica y visual, el modelo computacional más poderoso de la Teoría de Autómatas.

La implementación en Python demostró una correspondencia directa entre los componentes formales de una MT ($Q, \Sigma, \Gamma, \delta, q_0, B, F$) y las estructuras de datos del lenguaje python (str, set, dict, list). En particular, el uso de un diccionario con tuplas como llaves, nos permite representar la función de transición de manera eficiente.

Por último, la interfaz gráfica desarrollada con **Tkinter** facilita el uso del simulador, permitiendo ingresar cualquier MT y probar palabras de forma interactiva sin necesidad de modificar el código, además de que la visualización paso a paso de las descripciones instantáneas es especialmente útil para comprender el comportamiento interno de la máquina en cada transición.

6. Bibliografía

- De La Maza, M.E. – Material de clases INFO 139, Teoría de Autómatas, Universidad Austral de Chile.
- Hopcroft, J.E., Motwani, R., Ullman, J.D. – Introduction to Automata Theory, Languages, and Computation. 3rd ed. Pearson, 2006.
- Python Software Foundation – Documentación oficial de Python 3.
<https://docs.python.org/3/>
- Python Software Foundation – Módulo tkinter.
<https://docs.python.org/3/library/tkinter.html>
- Wikipedia – Turing machine. https://en.wikipedia.org/wiki/Turing_machine
- Repositorio del proyecto – <https://github.com/Fernando13423/Maquina-de-Turing>